

Compiler Construction Project Report

Spring 2026

BSCS-12B

Group Members

Muhammad Mahad (408576) • Abdul Moiz (375299) •
Muhammad Bashir Siddiqui (366006)

Modular Compiler for a Subset of C
Lexer, Parser, Semantic Analyzer, TAC & LLVM Backend

1. Introduction

This document presents the design and implementation of a modular compiler for a subset of the C programming language. The compiler is implemented entirely in Python and features Lexical Analysis, Syntax Analysis (Recursive Descent Parsing), Semantic Analysis (Symbol Table for scope checking), Code Optimization (Constant Folding and Dead Code Elimination), Intermediate Representation (TAC) generation, and finally LLVM IR Backend generation.

2. Subset Definition & Grammar

Features Supported

- Data Types: int, float, double
- Expressions: Arithmetic (+, -, *, /, %), Relational (==, !=, <, >, <=, >=), Logical (&&, ||, !)
- Control Flow: if-else, while, for loops
- Functions: Parameter passing and return statements
- Innovation Features: 1D Arrays, Constant Folding, Dead Code Elimination

Context-Free Grammar (Subset)

```
Program -> Decl*
Decl -> Type ID ( '(' Params ')' Block | '[' NUMBER ']' ';' | [
      '=' Expr ] ';' )
Block -> '{' Stmt* '}'
Stmt -> IfStmt | WhileStmt | ForStmt | ReturnStmt | ExprStmt |
Block
IfStmt -> 'if' '(' Expr ')' Stmt [ 'else' Stmt ]
WhileStmt -> 'while' '(' Expr ')' Stmt
ForStmt -> 'for' '(' ExprStmt Expr ';' Expr ')' Stmt
ExprStmt -> [ ID '=' Expr | ID '[' Expr ']' '=' Expr ] ';' | Expr
      ';'
Expr -> LogicalOr
```

3. Design of Compiler Phases

1. **Lexical Analyzer (lexer.py):** Tokenizes the input code using regular expressions to match keywords, numbers, identifiers, and symbols.
2. **Syntax Analyzer (parser.py):** Uses Recursive Descent Parsing to build an Abstract Syntax Tree (AST). It incorporates assignments directly into expressions to support standard C for loops.
3. **Semantic Analyzer (semantic.py):** Traverses the AST with a stack-based Symbol Table to enforce variable scoping, duplicate declarations, and array access checks.
4. **Optimizer (optimizer.py):** Performs Constant Folding on arithmetic expressions and eliminates dead code (e.g. if(0) blocks or while(0)).

5. **TAC Generator (tac.py):** A post-order AST traversal to generate flat Three-Address Code with explicit labels and temporaries.
6. **LLVM Backend (codegen.py):** Traverses the AST to generate standard LLVM Intermediate Representation (.ll). It handles memory allocation (alloca), loads, and stores to properly reflect local scoping.

4. Limitations

- Single-dimensional arrays only.
- Pointers and structs are not supported.
- Missing short-circuiting in logical operators at the LLVM level (currently evaluates both sides).
- Only simple typing (implicit promotion between int and float is limited).

5. Compilation Instructions

The compiler is designed to run in a standard Python 3 environment without external dependencies.

- Place all source files (lexer.py, parser.py, ast_nodes.py, semantic.py, optimizer.py, tac.py, codegen.py, main.py) in a directory.
- Run the compiler on a C source file: `python main.py input.c`
- The compiler will produce `output.tac` (Three-Address Code) and `output.ll` (LLVM IR).
- To compile the LLVM IR to a native executable, use clang: `clang output.ll -o output.exe`

6. Test Cases & Output

Test Case: Simple Arithmetic and Constant Folding (test1_arithmetic.c)

Source Code:

```
int main() {
    int a = 10;
    int b = 5 + 3;
    int c = a * b;
    return c;
}
```

Generated TAC:

```
func main:
begin_func
a = 10
b = 8
t1 = a * b
```

```
c = t1
return c
end_func
```

Generated LLVM IR:

; LLVM IR Generated by Custom Compiler

```
declare i32 @printf(i8*, ...)
```

```
define i32 @main() {
```

```
entry:
```

```
    %1 = alloca i32
```

```
    store i32 10, i32* %1
```

```
    %2 = alloca i32
```

```
    store i32 8, i32* %2
```

```
    %3 = alloca i32
```

```
    %4 = load i32, i32* %1
```

```
    %5 = load i32, i32* %2
```

```
    %6 = mul i32 %4, %5
```

```
    store i32 %6, i32* %3
```

```
    %7 = load i32, i32* %3
```

```
    ret i32 %7
```

```
    ret i32 0
```

```
}
```

Output:

```
PS C:\Users\Mahad-Lapi\Desktop\Compiler Construction\Assignments\Project\compiler> python main.py test1_arithmetic.c
--- Lexical Analysis ---

--- Syntax Analysis ---
AST generated successfully.

--- Semantic Analysis ---
Semantic analysis passed.

--- Optimization (Constant Folding & DCE) ---
Optimization complete.

--- TAC Generation ---

--- LLVM IR Generation ---

TAC written to output.tac
LLVM IR written to output.ll
```

Test Case: Loops and Dead Code Elimination (test2_loops.c)

Source Code:

```
int main() {
    int sum = 0;
    int i = 0;
    while (i < 10) {
        sum = sum + i;
        i = i + 1;
    }

    int j;
    for (j = 0; j < 5; j = j + 1) {
        sum = sum + j;
    }

    if (0) {
        sum = 0;
    }

    return sum;
}
```

Generated TAC:

```
func main:
begin_func
sum = 0
i = 0
L1:
t1 = i < 10
if_false t1 goto L2
t2 = sum + i
sum = t2
t3 = i + 1
i = t3
goto L1
L2:
j = 0
L3:
t4 = j < 5
if_false t4 goto L4
t5 = sum + j
sum = t5
t6 = j + 1
```

```
j = t6
goto L3
L4:
return sum
end_func
```

Generated LLVM IR:

; LLVM IR Generated by Custom Compiler

```
declare i32 @printf(i8*, ...)
define i32 @main() {
entry:
    %1 = alloca i32
    store i32 0, i32* %1
    %2 = alloca i32
    store i32 0, i32* %2
    br label %L1
L1:
    %3 = load i32, i32* %2
    %5 = icmp slt i32 %3, 10
    %4 = zext i1 %5 to i32
    %6 = icmp ne i32 %4, 0
    br i1 %6, label %L2, label %L3
L2:
    %7 = load i32, i32* %1
    %8 = load i32, i32* %2
    %9 = add i32 %7, %8
    store i32 %9, i32* %1
    %10 = load i32, i32* %2
    %11 = add i32 %10, 1
    store i32 %11, i32* %2
    br label %L1
L3:
    %12 = alloca i32
    store i32 0, i32* %12
    br label %L4
L4:
    %13 = load i32, i32* %12
    %15 = icmp slt i32 %13, 5
    %14 = zext i1 %15 to i32
    %16 = icmp ne i32 %14, 0
    br i1 %16, label %L5, label %L6
L5:
    %17 = load i32, i32* %1
    %18 = load i32, i32* %12
```

```

    %19 = add i32 %17, %18
    store i32 %19, i32* %1
    %20 = load i32, i32* %12
    %21 = add i32 %20, 1
    store i32 %21, i32* %12
    br label %L4
L6:
    %22 = load i32, i32* %1
    ret i32 %22
    ret i32 0
}

```

Output:

```

● PS C:\Users\Mahad-Lapi\Desktop\Compiler Construction\Assignments\Project\compiler> python main.py test2_loops.c
--- Lexical Analysis ---

--- Syntax Analysis ---
AST generated successfully.

--- Semantic Analysis ---
Semantic analysis passed.

--- Optimization (Constant Folding & DCE) ---
Optimization complete.

--- TAC Generation ---

--- LLVM IR Generation ---

TAC written to output.tac
LLVM IR written to output.ll

```

Test Case: Function Calls and Arrays (test3_functions_arrays.c)

Source Code:

```
int add(int x, int y) {
    return x + y;
}

int main() {
    int arr[5];
    arr[0] = 10;
    arr[1] = 20;

    int result = add(arr[0], arr[1]);
    return result;
}
```

Generated TAC:

```
func add:
begin_func
t1 = x + y
return t1
end_func

func main:
begin_func
alloc_array arr, 5
arr[0] = 10
arr[1] = 20
t2 = arr[0]
t3 = arr[1]
param t2
param t3
t4 = call add, 2
result = t4
return result
end_func
```

Generated LLVM IR:

```
; LLVM IR Generated by Custom Compiler
declare i32 @printf(i8*, ...)
define i32 @add(i32 %x_arg, i32 %y_arg) {
entry:
    %1 = alloca i32
```



```

    store i32 %x_arg, i32* %1
    %2 = alloca i32
    store i32 %y_arg, i32* %2
    %3 = load i32, i32* %1
    %4 = load i32, i32* %2
    %5 = add i32 %3, %4
    ret i32 %5
    ret i32 0
}

define i32 @main() {
entry:
    %1 = alloca i32, i32 5
    %2 = getelementptr i32, i32* %1, i32 0
    store i32 10, i32* %2
    %3 = getelementptr i32, i32* %1, i32 1
    store i32 20, i32* %3
    %4 = alloca i32
    %5 = getelementptr i32, i32* %1, i32 0
    %6 = load i32, i32* %5
    %7 = getelementptr i32, i32* %1, i32 1
    %8 = load i32, i32* %7
    %9 = call i32 @add(i32 %6, i32 %8)
    store i32 %9, i32* %4
    %10 = load i32, i32* %4
    ret i32 %10
    ret i32 0
}

```

Output:

```

PS C:\Users\Mahad-Lapi\Desktop\Compiler Construction\Assignments\Project\compiler> python main.py test3_functions_arrays.c
--- Lexical Analysis ---

--- Syntax Analysis ---
AST generated successfully.

--- Semantic Analysis ---
Semantic analysis passed.

--- Optimization (Constant Folding & DCE) ---
Optimization complete.

--- TAC Generation ---

--- LLVM IR Generation ---

TAC written to output.tac
LLVM IR written to output.ll

```